Reviewer Pack — Hook-Length Ratio Invariance in Symmetric Polynomial Decompo…

Entry cf7f1e6550cc · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 06:23:26 UTC

Hook-Length Ratio Invariance in Symmetric Polynomial Decompositions

Entry ID: `cf7f1e6550cc` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-20 06:23:26 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics **Field B** (complexity object): Extended Frege Proof Complexity

Statement:

> For a homogeneous polynomial f of degree n, let H(f) be the ratio of hook-lengths in its Young diagram decomposition under Schur-Weyl duality. Then H(perm_n) > H(det_m) for all m < n^{1.5}, and H(f) is invariant under linear substitutions preserving the polynomial's symmetry type.

Rationale (proposer's reasoning):

> Schur-Weyl duality provides a framework to decompose symmetric polynomials into irreducible representations, while Extended Frege complexity measures proof size. The hook-length ratio captures combinatorial structure preserved under linear transformations, offering a potential bridge between algebraic symmetry and proof complexity barriers.

Taxonomy category: `GCT_DET_PERM` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `2904ea103154885c`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.95	SAFE	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a*b) // gcd(a, b)

def factorial(n):
    if n == 0 or n == 1:
        return 1
    result = 1
    for i in range(2, n + 1):
        result *= i
    return result

def hook_length_formula(young_diagram):
    n = len(young_diagram)
    hook_lengths = []
    for row in range(n):
        for col in range(row + 1):
            hook_lengths.append((young_diagram[row][col] - col) * (n - row - young_diagram[row][co
    return math.prod(hook_lengths)

def perm_n_hook_length():
    n = 5
    hook_lengths = [i * (n - i) for i in range(n)]
    return math.prod(hook_lengths)

def det_m_hook_length(m):
    m = int(math.sqrt(m))
    if m == 0:
        return 1
```

```
    hook_lengths = [(m - col) * (m - row) for row in range(m) for col in range(row + 1)]
    return math.prod(hook_lengths)

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [5, 10, 15, 20, 30, 40]
    H_perm_n = perm_n_hook_length()
    H_det_m_values = []

    for n in n_values:
        if n < 5 or n > 40:
            continue
        H_det_m = det_m_hook_length(n)
        H_det_m_values.append(H_det_m)

        if H_det_m >= H_perm_n:
            return {
                "metric_name": "Hook-Length Ratio",
                "metric_value": H_det_m,
                "instances_tested": 1,
                "conjecture_holds": False,
                "counterexample": f"det_{n}={H_det_m}, perm_5={H_perm_n}"
            }

    return {
        "metric_name": "Hook-Length Ratio",
        "metric_value": H_det_m_values,
        "instances_tested": len(n_values),
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or list(range(2, 30))  # Default to first 29 primes if

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {{'seed': {seed}, ...{result}...}}")
        results.append(result)

    metric_values = [r["metric_value"] for r in results if isinstance(r["metric_value"], list)]
    mean = sum(metric_values) / len(metric_values)
    std_dev = math.sqrt(sum((x - mean) ** 2 for x in metric_values) / len(metric_values))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"det_m >= perm_5\" first_failing_seed={first_fai
    else:
        print("RESULT: INCONCLUSIVE insufficient_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
ed': 503, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1, 'conjectu
TRIAL: {'seed': 547, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 593, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 631, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 677, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 727, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 773, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 821, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 877, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1
TRIAL: {'seed': 929, ...{'metric_name': 'Hook-Length Ratio', 'metric_value': 8, 'instances_tested': 1

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_12f51e36.py", line 95, in <module>
    mean = sum(metric_values) / len(metric_values)
           ~~~~~~~~~~~~~~~~~~~^~~~~~~~~~~~~~~~~~~
ZeroDivisionError: division by zero
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> Test crashed before completing required validation; counterexample appears inconsistent with conjecture's parameters | next: Fix division-by-zero error in test code and re-run with proper parameter validation

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	42035
2	preregistration	ollama_remote	qwen3:8b	0	0	32116
3	novelty	ollama_remote	qwen3:8b	0	0	27336
4	novelty	ollama_remote	qwen3:8b	0	0	24697
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13688
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9813
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7202
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9922
9	judge	ollama_remote	qwen3:8b	0	0	18901

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 185709 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/cf7f1e6550cc.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/cf7f1e6550cc.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/cf7f1e6550cc.tar.gz` (if generated)